



PPE COMPLIANCE DETECTOR

Automated PPE Verification for Home Health Caregivers

ITAI 1378 — Final Presentation

MODEL TRAINED ✓ 90% mAP

TEAM MEMBERS

Huy Nguyen & Raymond Newton

PROJECT TIER

Tier 1

The Problem

- Home health caregivers are required to wear PPE (masks, gloves) during certain visits.
- Compliance today relies on self-reporting and occasional manual supervisor spot-checks.
- This does not scale across a large caregiver workforce and creates gaps during infection-control audits.

Who cares: Agency compliance officers, supervisors, and payer/UHC infection-control audits.

MANUAL CHECKS DON'T SCALE

1

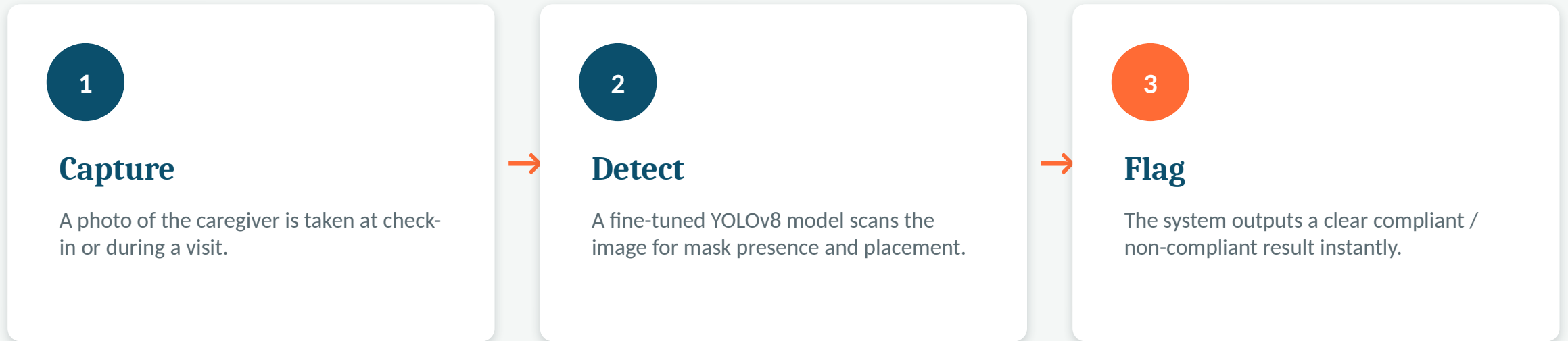
spot-check per visit — no continuous verification

Why it matters

Missed PPE compliance is both a health risk and a documentation liability during audits.

Our Solution

An object detection system that scans a photo of a caregiver and automatically flags PPE compliance — in real time, with no manual review.



Compliant → mask detected correctly **⚠ Non-Compliant** → mask missing or worn incorrectly

Technical Approach

TECHNIQUE	MODEL	FRAMEWORK
Object Detection Bounding boxes localize exactly where PPE is present or missing — not just a whole-image label.	YOLOv8n Fine-tuned for 75 epochs on free Colab GPU — fast to train, strong results for its size.	Ultralytics (PyTorch) Mature tooling, strong community support for PPE-style datasets, minimal boilerplate.

Why this stack: kept scope realistic for the timeline — real training and evaluation, without custom architecture design or manual labeling.

Data Plan

DATASET SIZE

4,547

labeled images

Source

Roboflow Universe
“Mask-Detection-YOLOv8” (AGH)

License: CC BY 4.0

Labels (3 classes)

with_mask

without_mask

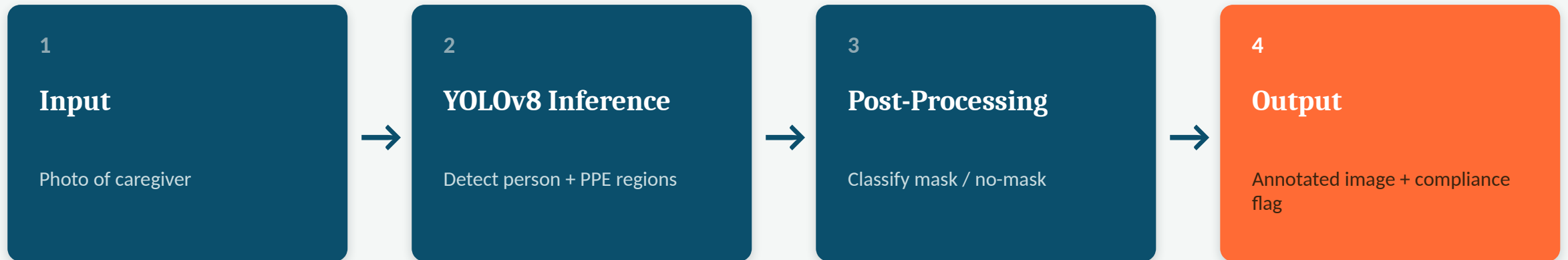
incorrectly_worn_mask

Core deliverable: with_mask / without_mask. Incorrect-mask detection is a stretch goal.

Preparation

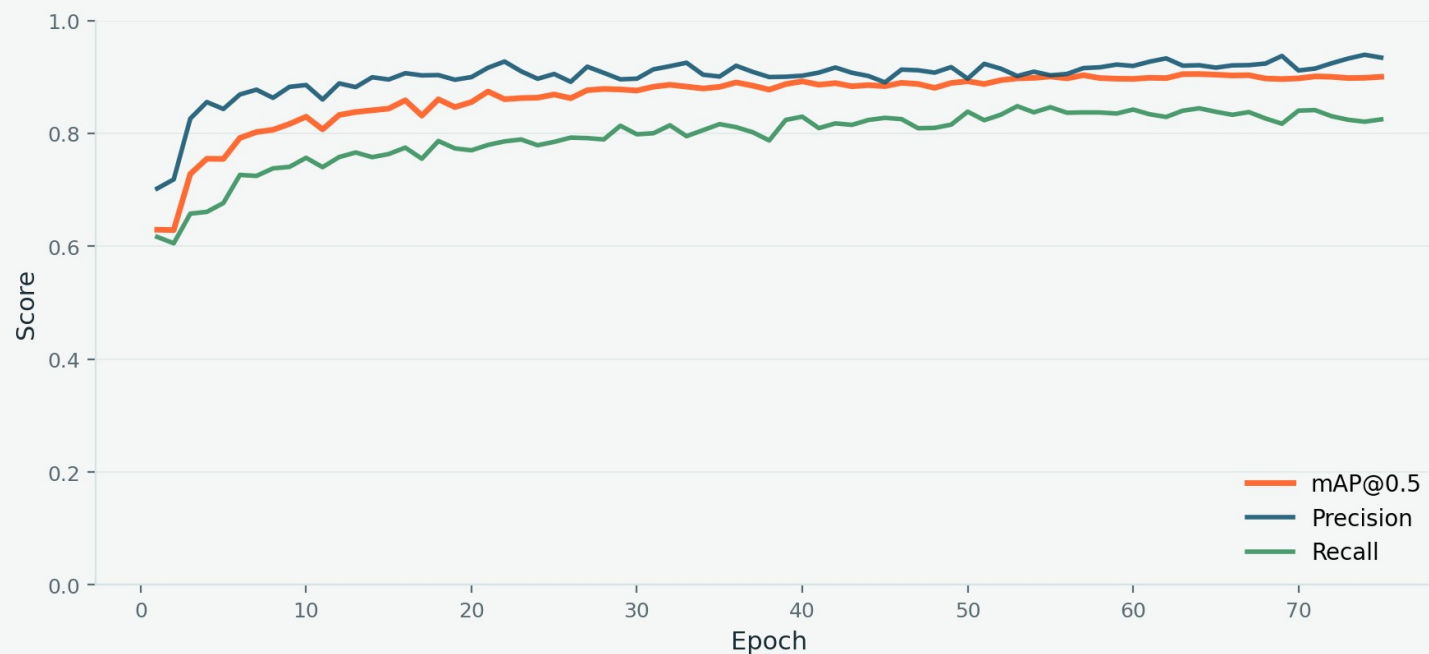
- No manual labeling — dataset ships pre-annotated in YOLO format.
- Exported directly via the Roboflow Python package.
- Standard Ultralytics augmentation applied at train time (flip, mosaic, brightness).

System Diagram



Pipeline is intentionally linear — one photo in, one clear compliance flag out. No manual review step required.

Training Results



Model exceeded our 75–80% mAP target, matching published benchmarks (~91%) for similar fine-tuned models on this dataset.

mAP@0.5

90.0%

Precision

93.4%

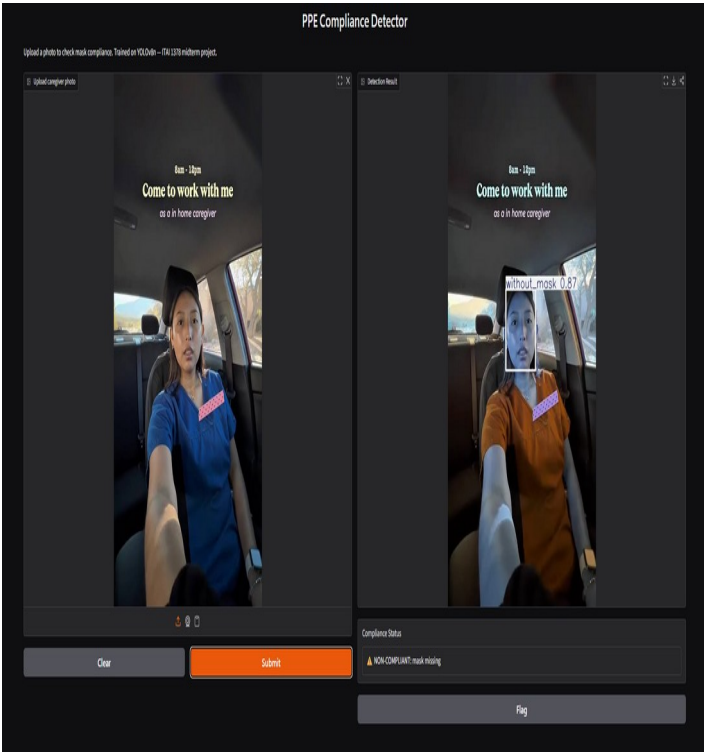
Recall

82.5%

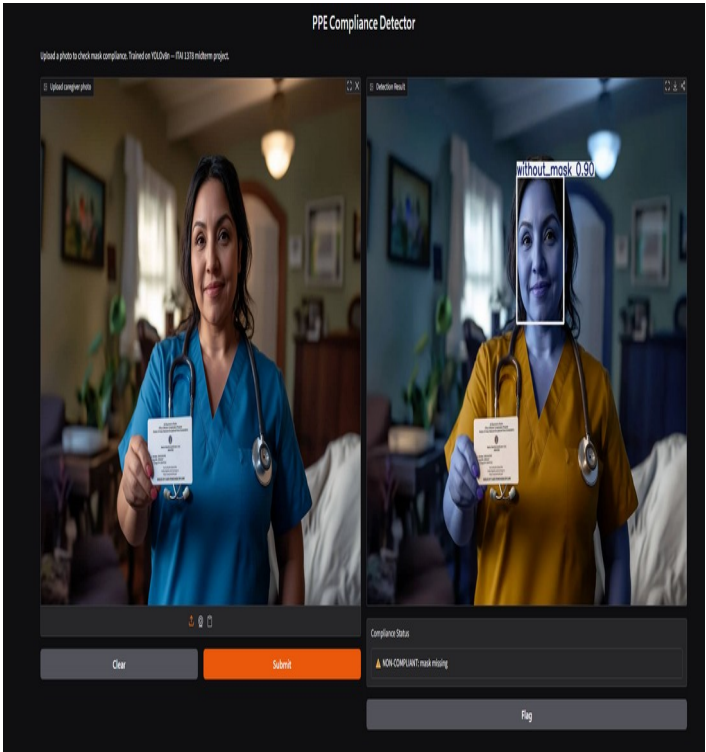
75 epochs • 76.4 min total training time on Colab T4 GPU

Live Demo

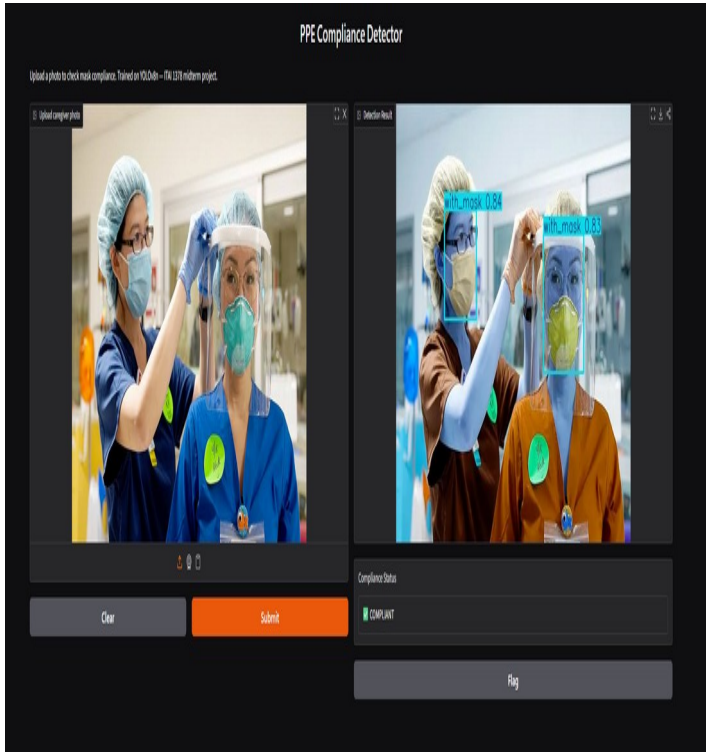
Interactive Gradio interface — upload a photo, get an instant compliance flag.



NON-COMPLIANT — mask missing (0.87)



NON-COMPLIANT — mask missing (0.90)



COMPLIANT — both masks detected (0.84, 0.83)

Success Metrics: Target vs. Actual

PRIMARY METRIC — mAP@0.5

Target: $\geq 75\text{--}80\%$

90.0%

✓ TARGET EXCEEDED

Best single checkpoint (epoch 64) hit 90.5% — model stayed stable through training.

SECONDARY METRIC — SPEED

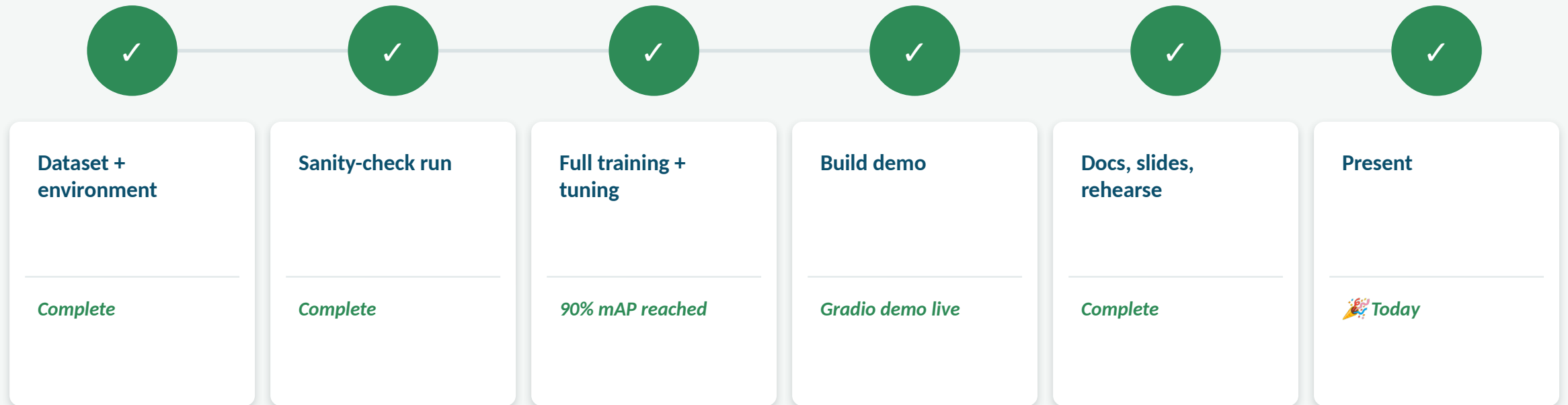
Target: $< 0.5\text{s} / \text{image}$

Precision 93.4%

Recall 82.5%

Confirmed fast, responsive inference during live Gradio demo — well under a second per image.

Project Timeline



Challenges & What We Learned

Challenge	How We Handled It
Full 75-epoch run took ~76 min	Used checkpointing (save_period=10) so progress was never lost to a Colab disconnect.
Domain mismatch (dataset ≠ real home visits)	Accepted as a known limitation — can't use real patient photos for privacy reasons.
Confirming target vs. actual model quality	Cross-checked our 90% mAP against a published benchmark on the same dataset (~91%) to validate the result was reasonable, not a fluke.
Turning raw detections into a usable signal	Built a simple compliance-flag function on top of raw YOLO output, then wrapped it in a Gradio UI for a real interactive demo.

Conclusion

COMPUTE USED

Colab T4 GPU

76.4 min total training

FRAMEWORKS

Ultralytics YOLOv8

PyTorch · Roboflow · Gradio

TOTAL COST

\$0

Free tiers covered the full project

A working PPE compliance detector — 90% mAP, real-time demo, built on public data and free compute.

Thank you — questions welcome